

ProjDesk Architecture & Command Philosophy

ProjDesk is not just a collection of scripts; it is an intelligent workspace manager built on a specific philosophy of human-computer interaction. This document outlines the core principles that dictate how commands are structured, how the CLI behaves, and how developers interact with it.

1. The Core Principle: Less Friction. More Code.

Every design decision in ProjDesk must serve one purpose: **reducing cognitive load and context switching**. The user should never have to remember *how* to start working. The tool should adapt to the user's intent, not the other way around.

2. The Semantic Command Tree (Progressive Refinement)

ProjDesk uses a "Progressive Refinement" command structure. Commands flow from left to right, going from broad intent to specific action.

- **Noun -> Verb -> Modifier** (or Action -> Target -> Detail)
- Each subsequent argument refines the previous one.

2.1. The "Alias as a Reward" System

ProjDesk rewards users who learn the tool by providing aggressive, single-letter aliases for almost every node in the command tree. The full words are for discovery; the aliases are for speed.

Examples of Progressive Refinement:

1. Opening a Project (The Default Action)

- `pd my-project` (Implicit intent: Open)
- `pd a my-project` (Refinement: Open/Create, specifically for Android)



2. Managing Recent Projects

- `pd recent` (Intent: Show/manage recent projects)
- `pd recent list` (Refinement: Specifically list them)
- *Reward:* `pd r ls`

3. Diagnostics and Fixing

- `pd doctor` (Intent: Run diagnostics on the workspace environment)
- `pd doctor fix` (Refinement: Run diagnostics and automatically fix issues)
- *Reward:* `pd dr f`

3. Modularity and Single Responsibility

The codebase in `src/` is strictly modular. No single file should exceed 150 lines if possible. Every file has one job.

- `init.sh` : The entry point. It registers the `pd` function in the user's shell and forwards all arguments to the router.
- `router.sh` (or `main` in `project.sh`): The brain. It parses the semantic tree (handling both full words and aliases) and dispatches to the correct module.
- `detect.sh` : The sensory organ. It inspects files (`build.gradle` , `docker-compose.yml`) to return boolean states.
- `project.sh` : The manipulator. It handles directory creation, navigation, and opening IDEs.

4. Design Guidelines for New Features

When contributing or adding new commands to ProjDesk, you must adhere to these rules:

1. **Never require a prompt if it can be automated:** If ProjDesk can detect the project type, it should not ask "Which IDE do you want?". It should just open it.
2. **Graceful Degradation:** If an automation fails (e.g., Docker isn't installed), catch the error, explain it in human-readable terms, and suggest the next best step.

3. **Silent Success, Loud Failure:** When `pd up` works, it shouldn't print a massive wall of text. It should start the containers and get out of the way. When it fails, it should be loud and clear.
4. **Always Provide an Alias:** Every new command (`plugin` , `template` , `update`) must come with a documented 1 or 2 letter alias.

5. The Command Map (Help Output)

This represents the mental model users should have of ProjDesk. This structure is mirrored in the `pd help` command.

```
pd [project-name]      -> Open or create a project (VS Code by default)
pd a [project-name]    -> Open or create an Android project (Android Studio)

# Environment Commands
pd up                  -> Start Docker containers
pd down                -> Stop Docker containers
pd rebuild             -> Rebuild Docker containers
pd logs                -> Tail container logs

# Workspace Commands
pd recent (r)          -> Open the most recently used project
pd recent list (r ls)  -> List the last 10 used projects
pd list (ls)           -> List all projects in the workspace

# Diagnostic Commands
pd doctor (dr)          -> Check system dependencies (Docker, Studio, etc.)
pd doctor fix (dr f)    -> Attempt to auto-fix missing dependencies

# System Commands
pd help (h)            -> Show this help menu
```